

Tutorato Programmazione 1

01/12/25

1.

Il file `wrong.cpp` contiene codice sbagliato, trovatene gli errori.

2.

Il file cerchi.txt e' formattato nel seguente modo:

x1 y1 r1

x2 y2 r2

. . .

xN yN rN

Dove (x_i, y_i) e' il centro dell'i-esimo cerchio e r_i e' il suo raggio.

Leggere il file e salvare ogni cerchio in una apposita struct Cerchio.

Trovare poi il numero di cerchi diversi che si intersecano, e stamparlo a video.
(numero di cerchi, non punti di intersezione)

(Non e' dato il numero di cerchi, ma supponete siano meno di 128)

3.

Il file rettangoli.txt e' formattato nel seguente modo:

```
x1_1 y1_1 x2_1 y2_1
x1_2 y1_2 x2_2 y2_2
. . .
x1_N y1_N x2_N y2_N
```

Dove $(x1_i, y1_i)$ e' l'angolo in basso a sinistra, e $(x2_i, y2_i)$ e' l'angolo in alto a destra dell'i-esimo rettangolo.

Leggere il file e salvare ogni rettangolo in una apposita struct Rettangolo.

Trovare il numero di rettangoli diversi che si intersecano.

(rettangoli, non punti d'intersezione)

Inoltre, trovare anche la superficie COMPLESSIVA totale.

(in pratica, se 2 rettangoli si intersecano, si somma l'area dei due e si sottrae l'area dell'intersezione)

4.

implementare un parser di URL.

Il file URL.txt contiene URL riga per riga formattati nel seguente modo:

```
protocollo://host[:porta][/percorso][?querystring]
```

dove il numero di porta, il percorso, e la querystring sono opzionali e non si escludono a vicenda.

La querystring, e' a sua volta formattata nel seguente modo:

```
?arg1=val1&arg2=val2...
```

Ed e' una sequenza di coppie nome,valore.

Implementare le struct:

```
Querystring {  
    char* arg[32];  
    char* val[32];  
}  
URL {  
    char* protocol, host, port, path;  
    Querystring qs;  
}
```

E due funzioni per parsare un URL (stringa letta dal file) e stampare solo i campi esistenti nelle struct querystring e URL.

Es:

<http://127.0.0.1:80>

protocol: http
host: 127.0.0.1
port: 80

<https://wikipedia.org/wiki/URL?a=ciao>

protocol: https
host: wikipedia.org
path: /wiki/URL
a : ciao

<http://192.168.1.100:8000?node=1&c=aaa>

protocol: http
host: 192.168.1.100
port: 8000
node: 1
c: aaa

5.

Implementare uno Stack con un array come visto a lezione.

Implementarne i metodi:

- void init(int* s)
- void deinit(int* s)
- bool push(int* s, int val)
- bool pop(int *s)
- bool top(int *s, int& val)

push, pop e top ritornano true se l'operazione ha successo, falso altrimenti.

6.

Implementare una Queue con un array come visto a lezione.

Implementarne i metodi:

- void init(int* q)
- void deinit(int* q)
- bool enqueue(int* q, int val)
- bool dequeue(int *q)
- bool first(int *q, int& val)

enqueue, dequeue e first ritornano true se l'operazione ha successo, falso altrimenti.